Enterprise EMS - Project Packages Guide

Welcome to the comprehensive packages and setup documentation guide for the **Enterprise Employee Management System (EMS)**. This document serves as a complete reference sheet listing every package, library, and devDependency configured in both the frontend (React.js) and backend (Node.js/Express.js) environments, including exact versions, usages, and project-specific examples.

This guide is designed for onboarding new developers, documentation logs, and academic or internship project submissions.

Table of Contents

1. Project Overview
2. Frontend Packages Reference
3. Backend Packages Reference
4. Installation & Setup Guide
5. Dependency Architecture Tree
6. Project Directory Layout
7. Command Reference Library
8. Unified Dependency Summary Table

Project Overview

- **Project Name:** Enterprise Employee Management System (EMS)
- **Technology Stack:** Full-stack JavaScript (PERN/MERN variation using MySQL)
- **Frontend Framework:** React.js (built on Vite for rapid compilation and HMR)
- **Backend Framework:** Node.js with Express.js (MVC architectural pattern)
- **Database:** MySQL (relational database engine managed via XAMPP / phpMyAdmin)
- **Authentication:** JSON Web Token (JWT) stateless auth + HTTP state contexts
- **Security layer:** bcrypt password hashing + helmet headers + CORS controls + validation middleware
- **API Architecture:** RESTful endpoints with JSON request/response payloads

---

Frontend Packages Reference

The frontend application uses modern React 18 libraries inside a Vite-powered project wrapper. Below is the list of all packages found in **frontend/package.json**:

Package Name	Version	Purpose	Why it is Used
`react`	`^18.3.1`	Core UI Engine	Provides component-based state, hooks, and virtual DOM mapping.
`react-dom`	`^18.3.1`	DOM Renderer	Handles virtual-to-real DOM updates in client browsers.
`react-router-dom`	`^7.18.1`	Single Page Routing	Governs secure client-side navigation, public/protected route filters.
`axios`	`^1.18.1`	HTTP Client Requests	Manages API communication, header injection, and data formatting.
`react-icons`	`^5.7.0`	UI Icons Library	Imports scalable vector icons from popular frameworks (Feather, FontAwesome).

Frontend Deep-Dive (Explanations & Usage Examples)

1. React (`^18.3.1`)

- **Short Explanation:** The foundation library for building user interfaces. It enables declarative component designs, hooks like `useState` and `useEffect` for reactive rendering, and manages DOM updates efficiently.
- **Where it is used:** Used universally in every file under `frontend/src/` (components, contexts, pages).
- **Example Usage:** Used in **Dashboard.jsx** to track filters and records state:

```
const [rawEmployees, setRawEmployees] = useState([]);
const [loading, setLoading] = useState(true);
```

2. React DOM (`^18.3.1`)

- **Short Explanation:** Provides DOM-specific methods that act as the entry point to bind the React application to the browser's document node (`#root`).
- **Where it is used:** Configured at the root bootstrap script **main.jsx**.
- **Example Usage:**

```
import { createRoot } from 'react-dom/client';
createRoot(document.getElementById('root')).render(<App />);
```

3. React Router DOM (`^7.18.1`)

- **Short Explanation:** Provides client-side navigation capabilities. It enables declarative routing without page refreshes, path parameters, and query reading.
- **Where it is used:** Configured inside **App.jsx** and read in sidebar links and view routing.
- **Example Usage:**

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
// Mapping routes
<Route path="/employees" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
```

4. Axios (`^1.18.1`)

- **Short Explanation:** A promise-based HTTP client for the browser and node.js. It simplifies requesting endpoints, automatically serializes JSON, and configures authorization tokens in request headers.
- **Where it is used:** Configured inside **api.js** and imported globally to make REST requests.
- **Example Usage:**

```
import axios from 'axios';
const api = axios.create({ baseURL: 'http://localhost:5000/api' });
```

5. React Icons (`^5.7.0`)

- **Short Explanation:** Integrates lightweight icon packages into React. Rather than downloading heavy font files, it imports specific vector icons as React components.
- **Where it is used:** Configured in Sidebar, Header, Modal Forms, and Dashboard buttons.
- **Example Usage:**

```
import { FiUsers, FiDollarSign, FiSearch } from 'react-icons/fi';
// Rendering icon tag
<div className="stat-icon"><FiUsers /></div>
```

Backend Packages Reference

The backend operates on a Node/Express server and hooks directly into MySQL. Below is the list of all packages in **backend/package.json**:

Package Name	Version	Purpose	Why it is Used
`express`	`^4.19.2`	Core Server Framework	Handles routing, request parses, controller pipelines, and middlewares.
`mysql2`	`^3.10.1`	Database Driver	Connects Node to MySQL, supports pools, and executes raw SQL queries.
`cors`	`^2.8.5`	Cross-Origin Filter	Unlocks API access requests for the frontend React client port.
`dotenv`	`^16.4.5`	Configuration Loader	Loads server environments (.env file values) into process variables.
`bcryptjs`	`^2.4.3`	Password Hashing	Secures administrator credentials using cryptographic bcrypt functions.
`jsonwebtoken`	`^9.0.2`	Token Authorization	Generates and decodes secure stateless JWT authorization headers.
`helmet`	`^7.1.0`	Header Security	Adds HTTP response headers to defend against security vulnerabilities.
`morgan`	`^1.10.0`	API Traffic Logger	Prints request methods, latency, and status codes to console streams.
`multer`	`^1.4.5-lts.1`	File Upload Parser	Handles multi-part form data uploads (such as avatars, documents).
`express-validator`	`^7.1.0`	Payload Validator	Evaluates incoming request parameters against sanitization rules.

Backend Deep-Dive (Explanations & Usage Examples)

1. Express (`^4.19.2`)

- **Short Explanation:** The light, unopinionated framework for hosting web servers in Node.js. It organizes API endpoints, serves static items, and chains controllers.
- **Where it is used:** Main starting pipeline in **server.js**.
- **Code Snippet:**

```
const express = require('express');
const app = express();
app.use(express.json());
app.listen(5000, () => console.log('Server running'));
```

2. MySQL2 (`^3.10.1`)

- **Short Explanation:** Connection driver enabling SQL commands in JS. It supports promise queries (`async/await`) and connection pooling to manage server loads.
- **Where it is used:** Initialized inside **db.js** and called in Models.
- **Code Snippet:**

```
const mysql = require('mysql2/promise');
const pool = mysql.createPool({ host: 'localhost', user: 'root', database: 'employee_management' });
const [rows] = await pool.query('SELECT * FROM employees');
```

3. CORS (`^2.8.5`)

- **Short Explanation:** Cross-Origin Resource Sharing middleware. By default, browsers prevent scripts from reading APIs hosted on other ports (e.g. React 5173 querying Node 5000). CORS overrides this block.
- **Where it is used:** Applied globally inside **server.js**.
- **Code Snippet:**

```
const cors = require('cors');
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
```

4. Dotenv (`^16.4.5`)

- **Short Explanation:** Loads configuration parameters from a private `.env` text file into the NodeJS execution context under `process.env`.
- **Where it is used:** Loaded at the very top of server initialization scripts.
- **Code Snippet:**

```
require('dotenv').config();
const port = process.env.PORT || 5000;
```

5. BcryptJS (`^2.4.3`)

- **Short Explanation:** Hashing algorithm wrapper in JS. It hashes administrator passwords with salts before database storage and evaluates incoming login keys.
- **Where it is used:** Used inside **db.js** for admin seeding and `authController.js` for checking logins.
- **Code Snippet:**

```
const bcrypt = require('bcryptjs');
const hash = await bcrypt.hash('adminpassword', 10);
const match = await bcrypt.compare('inputpassword', hash);
```

6. JSON Web Token (`^9.0.2`)

- **Short Explanation:** Signs secure tokens carrying payload information (ID, username) back to the client. This token is passed with subsequent requests to verify permissions.
- **Where it is used:** Issued on login in `authController.js`, verified inside `authMiddleware.js`.
- **Code Snippet:**

```
const jwt = require('jsonwebtoken');
const token = jwt.sign({ id: user.id }, process.env.JWT_SECRET, { expiresIn: '8h' });
```

7. Helmet (`^7.1.0`)

- **Short Explanation:** Security headers middleware. Sets diverse HTTP headers to block cross-site scripting (XSS), clickjacking, and mime-type sniffing attacks.
- **Where it is used:** Applied globally inside **server.js**.
- **Code Snippet:**

```
const helmet = require('helmet');
app.use(helmet());
```

8. Morgan (`^1.10.0`)

- **Short Explanation:** Logging server middleware. Standardizes console reports detailing incoming API routes, response times, HTTP statuses, and client browsers.
- **Where it is used:** Chained globally in `server.js` under developer mode settings.
- **Code Snippet:**

```
const morgan = require('morgan');
app.use(morgan('dev'));
```

9. Multer (`^1.4.5-lts.1`)

- **Short Explanation:** Middleware handling multi-part form payloads. It parses uploaded files (like avatars/resumes) and stores them to local storage folders.
- **Where it is used:** Integrated in upload routes configuration hooks.
- **Code Snippet:**

```
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });
app.post('/api/upload', upload.single('avatar'), (req, res) => res.json(req.file));
```

10. Express Validator (`^7.1.0`)

- **Short Explanation:** Validation framework middleware. Cleans up input elements to safeguard SQL queries, checks email formats, and rejects empty strings.
- **Where it is used:** Configured inside **validationMiddleware.js**.
- **Code Snippet:**

```
const { body, validationResult } = require('express-validator');
app.post('/user', [body('email').isEmail()], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });
});
```

Installation & Setup Guide

This section outlines how to set up, install, and execute the full-stack system from scratch on a new development environment.

Step 1: Install Node.js

Download and install Node.js (LTS version recommended) from the official website: https://nodejs.org/ (https://nodejs.org/). This will install both Node and NPM (Node Package Manager) on your machine.

Step 2: Clone the Project

Clone the repository files to your local workstation:

```
git clone <repository_url>
cd "employe management"
```

Step 3: Set Up Frontend Dependencies

Navigate to the frontend folder and download the packages:

```
cd frontend
npm install
```

Step 4: Set Up Backend Dependencies

Open a second terminal window, navigate to the backend folder, and install packages:

```
cd backend
npm install
```

Step 5: Install & Start MySQL using XAMPP

1. Download and install **XAMPP** from Apache Friends: https://www.apachefriends.org/ (https://www.apachefriends.org/).
2. Start the XAMPP Control Panel.
3. Turn on the **Apache** and **MySQL** modules by clicking "Start" next to them.

Step 6: Import SQL Database

1. Open your browser and go to: `http://localhost/phpmyadmin/`.
2. Click **New** in the left sidebar, type `employee_management` under database name, and select `utf8mb4_general_ci`.
3. Select the newly created database, go to the **Import** tab at the top.
4. Click **Choose File** and locate **employee_management.sql** in the project root folder.
5. Click **Import** at the bottom of the page to import the database.

Step 7: Configure Environment Variables (.env)

Create a `.env` file inside the `backend/` directory and configure the environment values:

```
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASS=
DB_NAME=employee_management
JWT_SECRET=supersecure_enterprise_ems_secret_token_key_101
```

Step 8: Run the Backend Server

Inside the `backend/` terminal directory, execute the development command:

```
npm run dev
```

Expected log output:

```
Database Connected (Server connection established)
Database "employee_management" ensured
Database tables verified/created successfully
Server Started: Express server running on port 5000
```

Step 9: Run the Frontend Server

Inside the `frontend/` terminal directory, execute the development command:

```
npm run dev
```

Expected log output:

```
  VITE v5.4.21  ready in 437 ms
  ➜  Local:   http://localhost:5173/
```

Step 10: Open in Browser

Open your browser and navigate to: `http://localhost:5173/`.

- **User ID:** `admin`
- **Password:** `admin123`

Dependency Architecture Tree

The visual flowchart below illustrates how the primary npm dependencies relate to one another in the application's lifecycle:

```
React (Frontend)
│
├── React Router (v7)
├── Axios Client
└── React Icons

Node/Express (Backend)
│
├── Express Framework
├── MySQL2 Connector
├── JWT Security (jsonwebtoken)
├── BcryptJS encoding
├── Morgan Logger
└── dotenv Loader
```

Project Directory Layout

The directory structure follows the standard Model-View-Controller (MVC) organization:

```
employe management/
├── backend/                  # Node.js backend workspace
│   ├── src/
│   │   ├── config/           # Database configuration and tables setup
│   │   ├── controllers/      # Route controllers (Auth controls, CRUD methods)
│   │   ├── middleware/       # Token authorization and express validator rules
│   │   ├── models/           # MySQL query models (User model, Employee model)
│   │   ├── routes/           # REST endpoint mapping
│   │   └── server.js         # Entry point for backend Express app
│   ├── .env                  # Port, database configuration details (Ignored in git)
│   └── package.json          # Backend dependencies registry
│
├── frontend/                 # Vite + React frontend workspace
│   ├── src/
│   │   ├── components/       # Layout blocks (Sidebar, Header navbar, Forms modals)
│   │   ├── context/          # State auth container (AuthContext)
│   │   ├── pages/            # Login page, Forgot password, Dashboard layouts
│   │   ├── utils/            # Axios default configurations (api.js)
│   │   ├── App.jsx           # Application main router paths
│   │   ├── main.jsx          # DOM entry point
│   │   └── index.css         # Global style design directives (Themes, layouts)
│   ├── index.html            # Main site HTML frame
│   └── package.json          # Frontend dependencies registry
│
├── employee_management.sql   # Backed-up database schema dump
├── README.md                 # Brief introduction guide
└── Project_Packages_Guide.md # Comprehensive packages guide (This file)
```

Command Reference Library

Below is a reference guide listing every command configured inside the package json files:

1. `npm install`

- **Description:** Scans the folder's `package.json` file, looks up all dependencies, and downloads them from the registry to a local `node_modules/` folder.
- **When to run:** Run this once right after cloning the project files or when adding new libraries.

2. `npm run dev`

- **Description:** Launches the workspace in development mode.
    - *Frontend:* Fires up the Vite server (with hot reload enabled).
    - *Backend:* Starts nodemon to monitor source code file edits and automatically restarts the process upon saving changes.
- **When to run:** Run this continuously during local development.

3. `npm run build`

- **Description:** Triggers Vite to compile, bundle, and optimize the React code into static assets (HTML, CSS, JS) saved inside a `/dist` production directory.
- **When to run:** Run this before deploying the application online.

4. `npm start`

- **Description:** Starts a node process execution pointing directly to the compiled script (`node src/server.js`), without monitoring file modifications.
- **When to run:** Used by production environments (like Heroku or Docker containers) to host the backend.

5. `npm audit`

- **Description:** Scans the installed packages list, evaluates them against the npm security database, and lists vulnerability disclosures.
- **When to run:** Run periodically to monitor security compliance.

6. `npm update`

- **Description:** Upgrades the installed libraries inside the semantic range listed in `package.json` to their newest releases.
- **When to run:** Run to obtain patch fixes and non-breaking version improvements.

Unified Dependency Summary Table

The table below lists the essential packages across both projects in a unified format:

Package	Purpose	Required?	Module
`react`	Component building interface	**Yes**	Frontend
`react-router-dom`	Page navigation	**Yes**	Frontend
`axios`	Back-and-forth REST API calls	**Yes**	Frontend
`react-icons`	Responsive graphics styling	**Yes**	Frontend
`express`	REST server routing	**Yes**	Backend
`mysql2`	SQL database queries	**Yes**	Backend
`cors`	Cross-origin access allowance	**Yes**	Backend
`dotenv`	Server variables loading	**Yes**	Backend
`bcryptjs`	Password security encoding	**Yes**	Backend
`jsonwebtoken`	Token user session tracking	**Yes**	Backend
`helmet`	HTTP header protection	**Yes**	Backend

`morgan`	Performance log reports	**Yes**	Backend
`multer`	Upload streams handling	**Yes**	Backend
`express-validator`	Request parameter scanning	**Yes**	Backend
`nodemon`	Dev server auto-restart	No (Dev Only)	Backend
`vite`	Project compiler	No (Dev Only)	Frontend

This guide was generated automatically by scanning the source directory configuration files for the Enterprise Employee Management System (EMS).